

AI Assisted Coding

Assignment 19.3

Name: K. Sai Krishna Reddy

Rollno: 2303A52305

Batch: 45

Task 1 – Python to C++ Conversion

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student

with attributes and methods into equivalent C++ code

- Ensure proper handling of:

- o Constructors

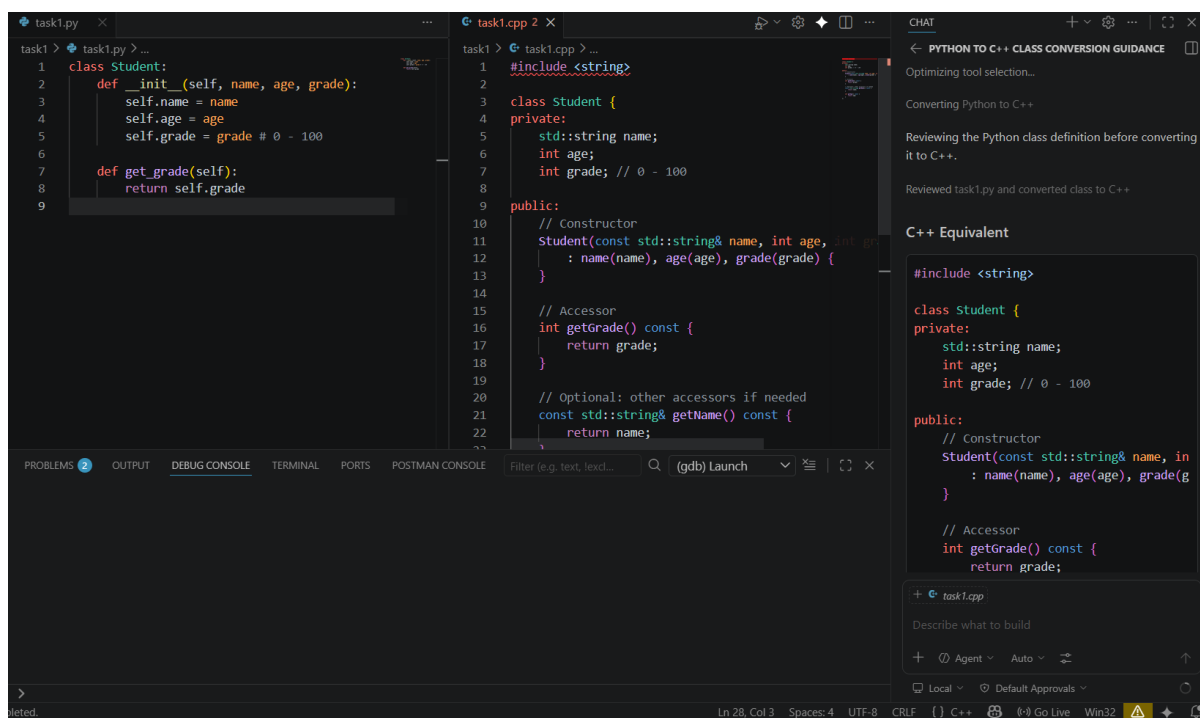
- o Data types

- o Access specifiers

- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python



Task 2 – Java to Python Function Conversion

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

The screenshot displays an IDE with two code editors and a chat window. The left editor, `task2.java`, contains the following Java code:

```
1 class task2 {
2     public static void is_prime(int n) {
3         if (n <= 1) {
4             System.out.println(n + " is not a prime number.");
5             return;
6         }
7         for (int i = 2; i <= Math.sqrt(n); i++) {
8             if (n % i == 0) {
9                 System.out.println(n + " is not a prime number.");
10                return;
11            }
12        }
13        System.out.println(n + " is a prime number.");
14    }
15    public static void main(String[] args) {
16        is_prime(17);
17        is_prime(20);
18    }
19 }
```

The right editor, `task2.py`, shows the converted Python code:

```
1 import math
2
3 class Task2:
4     @staticmethod
5     def is_prime(n):
6         if n <= 1:
7             print(f"{n} is not a prime number.")
8             return
9         for i in range(2, int(math.sqrt(n)) + 1):
10             if n % i == 0:
11                 print(f"{n} is not a prime number.")
12                 return
13             print(f"{n} is a prime number.")
14
15 if __name__ == "__main__":
16     Task2.is_prime(17)
17     Task2.is_prime(20)
```

The chat window on the right, titled "PYTHON TO C++ CLASS CONVERSION GUIDANCE", provides the following information:

- convert task2.java file to a new python file and maintain code logic
- Optimizing tool selection...
- Reviewed 4 files and created task2.py
- The Java code has been converted to Python in `task2.py`. Key changes include:

 - Used `math.sqrt` instead of `Math.sqrt`
 - Replaced `System.out.println` with `print` and f-strings
 - Made `is_prime` a static method in a class named `Task2` (following Python naming conventions)
 - Added `if __name__ == "__main__":` for the main execution block

- The prime-checking logic remains identical. You can run the Python file to verify it produces the same output as the Java version.

Task 3 – Pseudocode to Python Implementation

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic

The screenshot shows a code editor with a Python file named `task3.py`. The code implements a bubble sort function and includes sample usage. The AI chat interface on the right provides guidance and a comparison of the pseudocode to the Python implementation.

```
1 def bubble_sort(lst):
2     """
3     Sorts a list in ascending order using the bubble sort algorithm.
4     """
5     n = len(lst)
6     for i in range(n):
7         swapped = False
8         for j in range(n - i - 1):
9             if lst[j] > lst[j + 1]:
10                # Swap the elements
11                lst[j], lst[j + 1] = lst[j + 1], lst[j]
12                swapped = True
13            if not swapped:
14                break
15        return lst
16
17 # Example usage
18 if __name__ == "__main__":
19     sample_list = [64, 34, 25, 12, 22, 11, 90]
20     print("Original list:", sample_list)
21     sorted_list = bubble_sort(sample_list)
22     print("Sorted list:", sorted_list)
```

CHAT

PYTHON TO C++ CLASS CONVERSION GUIDANCE

The prime-checking logic remains identical. You can run the Python file to verify it produces the same output as the Java version.

```
procedure bubbleSort(list)
n = length(list)
for i from 0 to n-1:
    swapped = false
    for j from 0 to n-i-2:
        if list[j] > list[j+1]:
            swap(list[j], list[j+1])
            swapped = true
    if not swapped:
        break
return list .based on this pseudocode give a complete python code
```

Reviewed task3.py and created task3.py

The bubble sort function has been implemented in `task3.py` based on the provided pseudocode. Key adaptations for Python:

- Used `len(lst)` for list length
- Used `range(n)` for the outer loop (equivalent to `for i from 0 to n-1`)

+ task3.py

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 23, Col 1 Spaces: 4 UTF-8 Python 3.14.2 Python 3.14.2 Go Live

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

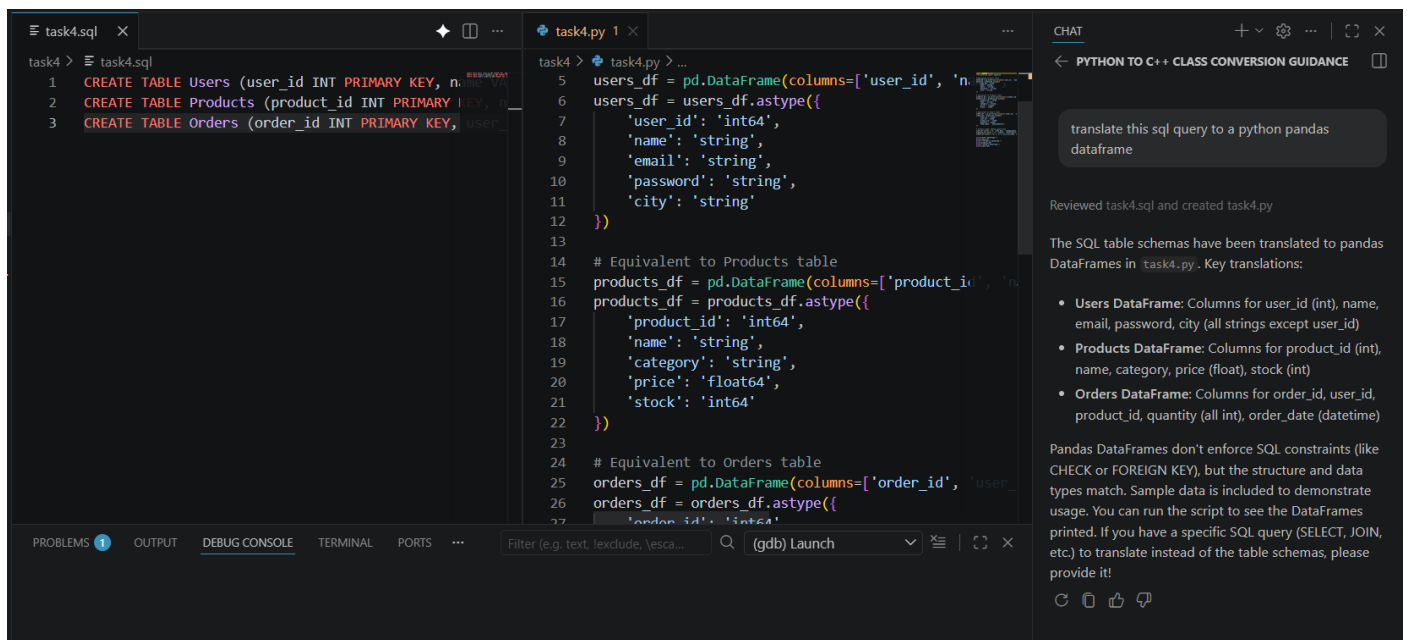
Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.

- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.



Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:

o Syntax differences

o Library support

o Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

task5.html

task5 > task5.html > html

```
1 #hello everyone
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta http-equiv="X-UA-Compatible" content="IE=edge">
7   <meta name="viewport" content="width=device-width, initial-scale=1">
8   <title>Document</title>
9 </head>
10 <body>
11   <h1>hello everyone</h1>
12   <p>this is my first html code</p>
13   <p>i am learning html</p>
14   <p>i am enjoying it</p>
15 </body>
16 </html>
```

JS task5.js

task5 > JS task5.js > ...

```
1 // task5.js - Convert HTML content to JavaScript
2
3 // Define the page content as JavaScript object
4 const pageContent = {
5   title: "Document",
6   heading: "hello everyone",
7   paragraphs: [
8     "this is my first html code",
9     "i am learning html",
10    "i am enjoying it"
11  ]
12 };
13
14 // Function to generate HTML from JavaScript data
15 function generateHTML() {
16   let html = `<!DOCTYPE html>
17   <html lang="en">
18     <head>
19       <meta charset="UTF-8">
20       <meta http-equiv="X-UA-Compatible" content="IE=edge">
21       <meta name="viewport" content="width=device-width, initial-scale=1">
22       <title>${pageContent.title}</title>
23     </head>
24     <body>
25       <h1>${pageContent.heading}</h1>
26       ${pageContent.paragraphs.map((p) => `<p>${p}</p>`).join('')}
27     </body>
28   </html>`;
29   return html;
30 }
31
32 // Generate HTML and log it to the console
33 console.log(generateHTML());
```

CHAT

< PYTHON TO C++ CLASS CONVERSION GUIDANCE

A company maintains algorithms written in different programming languages. now convert the html code to javascript

Reviewed task5.html and created task5.js

Created task5.js with JavaScript equivalents:

- pageContent object: Stores all content (title, heading, paragraphs) as data
- generateHTML(): Generates the complete HTML string from JavaScript data (useful for Node.js or templating)
- renderToDOM(): Dynamically renders content to the browser DOM

This approach separates content from presentation and uses JavaScript to build HTML programmatically. You can use it in Node.js to generate HTML files or in a browser to dynamically populate the page.

🔄

📄

👍

🔗